



CLAUDE CODE ADVANCED: THE BROWSER, THE PHONE, THE COMPANY

Claude in Chrome

Claude Code Advanced, episode 1

It shares your login state. It never authenticated as you, and it will not.

The code compiles, the tests pass, the diff looks right, and you still alt-tab to a browser and click around for thirty seconds to see whether the UI works. Thirty seconds, forty times a day.

Claude in Chrome closes that loop. This sheet is what it can do, what it asks permission for, and the preconditions that will otherwise cost you an evening before you get it running at all.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Setup: `--chrome`, `/chrome`, and enabling it by default
- The borrowed login session, and what borrowed means
- UI and end-to-end testing, which is the strongest use
- Live debugging from console and DOM state
- File uploads and their three restrictions
- Browser tools in plan mode: what runs free and what asks
- Every documented limit, dated
- The three failures you will actually hit

Out of scope

- The Claude in Chrome extension's own features, which are separate
- Replacing your test suite, which this does not do
- Any workflow that needs it to log in for you, which it will not
- Native desktop apps, which is computer use, a different page

Read the limits table BEFORE the capabilities list. This feature is easy to use and comparatively hard to turn on.

01 Preconditions and limits, all checked September 2026

Plan required	a direct Anthropic plan: Pro, Max, Team or Enterprise
Bedrock, Google Cloud, Foundry	NOT available, a separate account is needed
Authentication	/login only. API key or long-lived token keeps it off
Windows Subsystem for Linux	not supported
Extension version	1.0.36 or higher
Browsers	Chrome, Edge, and Chromium browsers such as Brave, Arc, Vivaldi, Opera
Enabled by default	increases context usage, browser tools always loaded
Single upload size	10 MB total

These are the documented requirements, not opinions. Any one of them missing means the integration does not run, usually with an error that does not obviously name the cause.

03 Setup, in two commands

Start with `claude --chrome`. The first launch shows a one-time dialog explaining how site permissions work.

Run `/chrome` at any time to check the connection status, manage permissions, reconnect the extension, or choose which connected browser to drive when more than one is available. It is working when the panel shows "Status: Enabled" and "Extension: Installed".

To skip the flag every session, run `/chrome` and select "Enabled by default". Read the note attached to that setting before you do: it increases context usage because the browser tools are always loaded, so if you notice context climbing, turn it off and use the flag only when you need it.

In the VS Code extension, Chrome is available whenever the extension is installed, with no flag.

Run `/chrome` at any time to check the connection status, manage permissions, reconnect the extension, or choose which connected browser to use.

Claude Code docs, Chrome, checked September 2026

Enabling Chrome by default in the CLI increases context usage since browser tools are always loaded.

Claude Code docs, Chrome, checked September 2026

04 The borrowed session, which is the whole feature

Claude opens new tabs for browser tasks and shares your browser's login state, so it can access any site you are already signed into. Gmail, Notion, Google Docs, your company's admin panel, with no API connectors.

Read the verb carefully. It SHARES. It did not authenticate as you and it will not. When it encounters a login page or a CAPTCHA, it pauses and asks you to handle it manually.

That is the design rather than a missing feature, and it is the correct design: an agent that could complete a login flow on your behalf would be an agent that could be talked into doing so by a web page.

Keep the shape of it in mind, because it is the hinge of the next two episodes: the session is borrowed from a browser you are sitting in front of.

Claude opens new tabs for browser tasks and shares your browser's login state, so it can access any site you are already signed into.

Claude Code docs, Chrome, checked September 2026

When Claude encounters a login page or CAPTCHA, it pauses and asks you to handle it manually.

Claude Code docs, Chrome, checked September 2026



05 What it is actually for

TESTING WHAT YOU JUST BUILT is the strongest use and the one that pays for the setup on the first day. "I just updated the login form validation. Open localhost:3000, try submitting the form with invalid data, and check the error messages appear correctly."

LIVE DEBUGGING: it reads console errors and DOM state directly, then fixes the code that caused them. Tell it what pattern to look for rather than asking for all the console output, because logs are verbose and every line costs context.

DESIGN VERIFICATION: build the UI from a mock, open it, verify it matches.

Then the long tail: visual regressions, user flows, data extraction to a CSV, repetitive form filling from a local file, multi-site workflows across tabs, drafting straight into a web app you are signed into, and recording an interaction as a GIF.

The GIF carries a warning worth repeating: the recording captures everything visible in the browser, including account details on logged-in pages, so review it before sharing it outside your team.

Tell Claude what patterns to look for rather than asking for all console output, since logs can be verbose.

Claude Code docs, Chrome, checked September 2026

The recording captures everything visible in the browser, including account details on logged-in pages, so review it before sharing it outside your team.

Claude Code docs, Chrome, checked September 2026



06 Plan mode: reading is free, changing asks

In plan mode, browser tool calls that only read the page or browser state run without a permission prompt, and calls that change state prompt for approval.

READ-ONLY: `read_page`, `get_page_text`, `find`, reading console messages or network requests, and taking a screenshot. STATE-CHANGING: clicks, typing, navigation, tab and window management, and recording a GIF.

The detail that shows the model was thought through: an otherwise read-only call ALSO prompts when it sets a state-changing input flag, such as `createIfEmpty` on the tab context call, `clear` on the console and network readers, or `save_to_disk` on a screenshot. And a `browser_batch` call runs without a prompt only when every action inside it is read-only.

Underneath all of that, site-level permissions are inherited from the Chrome extension, so which sites Claude may browse, click and type on is managed in the extension settings.

In plan mode, browser tool calls that only read the page or browser state run without a permission prompt, and calls that change state prompt for approval.

Claude Code docs, Chrome, checked September 2026

A `browser_batch` call runs without a prompt only when every action inside it is read-only.

Claude Code docs, Chrome, checked September 2026

Site-level permissions are inherited from the Chrome extension.

Claude Code docs, Chrome, checked September 2026



07 Uploads, and what happens to the tabs

Claude can attach files from your machine to upload fields, and it works in remote sessions too because Claude Code reads the file and sends its contents to the browser.

Three restrictions. PERMISSIONS: it can upload a file only when the session is allowed to read it, so a permission rule denying Read also blocks the upload. SIZE: 10 MB of files in total for a single upload. HARD LINKS: it refuses files with multiple hard links, which is common inside package-manager stores like node_modules. Copy the file and upload the copy.

The tabs Claude opens are collected into a Chrome tab group tied to your session. Typing /clear closes that group, open pages included, unless work that survives the clear is still running. Switching sessions with /resume, exiting, or clearing while surviving work runs closes the group only if it holds nothing but empty new tabs, so a page you may still be reading stays open.

Claude can upload a file only when the session is allowed to read it; a single upload can include up to 10 MB of files in total; Claude refuses files that have multiple hard links.

Claude Code docs, Chrome, checked September 2026

The extension collects the tabs Claude opens into a Chrome tab group tied to your session.

Claude Code docs, Chrome, checked September 2026



08 The three failures you will actually hit

THE BROWSER STOPS RESPONDING ENTIRELY. Check for a modal dialog. A JavaScript alert, confirm or prompt blocks browser events and prevents Claude from receiving commands at all. Dismiss it manually, then tell Claude to continue.

IT WORKED FOR AN HOUR AND THEN STOPPED. The extension's service worker can go idle during extended sessions, which breaks the connection. Run `/chrome` and select "Reconnect extension". The error "Receiving end does not exist" is this.

IT WAS NEVER DETECTED. Check the extension is installed and enabled in `chrome://extensions`, check Chrome is running, then RESTART CHROME ONCE. The first time you enable the integration, Claude Code installs a native messaging host configuration file, and Chrome reads that file at startup.

One more worth knowing for corporate networks: "Browser extension is not connected" can also mean an organization IP allowlist is rejecting the connection to the bridge host, which is a network problem rather than an extension problem.

JavaScript dialogs block browser events and prevent Claude from receiving commands.

Claude Code docs, Chrome, checked September 2026

The Chrome extension's service worker can go idle during extended sessions, which breaks the connection. Run `/chrome` and select Reconnect extension.

Claude Code docs, Chrome, checked September 2026

Chrome reads the native messaging host configuration file on startup, so restart Chrome if the extension is not detected on the first attempt.

Claude Code docs, Chrome, checked September 2026



09 The honest framing

A browser is a stateful, slow, flaky dependency, and an agent driving one inherits all three of those properties.

What it is superb at is the loop between a change and its effect: I changed this, show me what happened. That loop previously required a person, and a tool that can close it is a genuine shift in how a working day feels.

What it is not is a replacement for a test suite. A suite is fast, deterministic, and runs in CI with no browser, no login, and no service worker that has to stay awake. If you are using Claude in Chrome to verify the same thing for the third time, you have not found a workflow. You have found a missing test.

And if the task genuinely cannot be done in a browser, the documentation points at computer use, which drives native applications instead.

Computer use controls native macOS apps when a task cannot be done in a browser.

Claude Code docs, Computer use, checked September 2026

A browser is a stateful, slow, flaky dependency, and an agent driving one inherits all three. This is a judgement about how to use the feature, not a documented claim.

Operational practice, not vendor documentation

Before you start, and while you work

1. Direct Anthropic plan, not a third-party provider?
2. Signed in with /login rather than an API key or long-lived token?
3. Extension 1.0.36 or higher, installed and enabled?
4. Not on WSL?
5. Does /chrome show Status: Enabled and Extension: Installed?
6. If you enabled it by default, is your context usage still reasonable?
7. Are site permissions in the extension set to only what this task needs?
8. Uploading? Under 10 MB, readable by the session, not a hard-linked file?
9. Recording a GIF? Have you checked what is visible before it leaves your team?
10. Verified the same thing three times now? Write the test instead

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

Use Claude Code with Chrome

<https://code.claude.com/docs/en/chrome>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.